RUCHE · LEÇON

Leçon 6 — La mise en ligne

Généré le 13/10/2026

Jusqu'ici, Waiggle ne vivait que sur ton ordinateur. Pour que tes proches s'en servent, il faut un serveur allumé jour et nuit, joignable par un nom (waiggle.io), protégé, sauvegardé et facile à mettre à jour. Cette leçon suit le chemin d'une version, de la fusion d'une pull request jusqu'à l'écran de ta tante, puis montre les protections posées en route.

---

1. Le chemin d'une version, de bout en bout

1. Tu fusionnes une pull request sur main.
2. La CI de GitHub refait toutes les vérifications. Si tout est vert, elle **construit les images Docker** de l'API et de la porte d'entrée web, et les publie sur ghcr.io avec une **attestation de provenance**.
3. Tu demandes à Claude Code, sur ton PC : « déploie ». Il lance `infra/deploy-from-pc.sh`, qui vérifie l'attestation de chaque image, puis demande au serveur, par SSH, de lancer `infra/deploy.sh`.
4. Le serveur sauvegarde la base, télécharge les images, relance les conteneurs et vérifie que l'API répond avec la bonne version.
5. Ta tante tape waiggle.io. Le **DNS** traduit ce nom en adresse IP. Son navigateur ouvre une connexion **HTTPS** avec **Caddy**, qui lui envoie les pages de l'appli. L'appli appelle ensuite api.waiggle.io, que Caddy transmet à l'API.

```
Internet ──443──▶ web (Caddy + appli construite) ─┬─ waiggle.io      → les pages de
l'appli
                                                  └─ api.waiggle.io  → api:3000 ─▶ db
(réseau interne)
```

---

2. Le DNS : l'annuaire d'Internet

Les ordinateurs se trouvent par des adresses IP (`51.83.x.x`, ou en IPv6 `2a02:…`), les humains par des noms. Le **DNS** fait le lien. Dans la zone DNS de waiggle.io, tu crées des « enregistrements » :

- **A** et **AAAA** : « waiggle.io, c'est telle adresse IPv4, telle adresse IPv6 » ;
- **CNAME** : « www.waiggle.io, c'est comme waiggle.io » ;
- **CAA** : « seul Let's Encrypt a le droit de fabriquer des certificats pour ce domaine » ;

- **TXT** et **CNAME** de Brevo : la preuve que Brevo a le droit d'envoyer des e-mails au nom de waiggle.io (DKIM, DMARC). Sans eux, nos e-mails finiraient dans les indésirables.

> **Analogie**
>
> L'annuaire téléphonique : on cherche « Boulangerie Martin », on obtient un numéro. Les enregistrements CAA et DKIM, ce sont les tampons officiels qui disent « ce courrier vient vraiment de chez nous ».

---

3. Caddy, la porte d'entrée

Caddy est le seul programme joignable depuis Internet. Il fait trois choses :

1. **Le HTTPS** : il obtient tout seul un certificat chez Let's Encrypt, et le renouvelle avant qu'il expire. Le cadenas du navigateur, c'est lui.
2. **Servir l'appli web** : l'appli est construite à l'avance en pages HTML. Caddy trouve la bonne page pour chaque adresse.
3. **Transmettre** les requêtes d'api.waiggle.io à l'API.

Les pages à paramètre posent un petit problème : il existe une page `projects/[id].html`, mais des milliers d'adresses `/projects/<numéro>`. Caddy les réécrit :

infra/Caddyfile (extrait)

```
    route {
        # 0. Le plan du site d'Expo Router (/_sitemap) est un outil de développement :
en ligne,
        #    il est « introuvable », comme le catalogue des composants.
        @sitemap path /_sitemap /_sitemap.html
        error @sitemap 404
        # 1. Le fichier lui-même (/_expo/static/js/…), ou la page du même nom :
        #    /sign-in → sign-in.html, /account/delete → account/delete.html.
        try_files {path} {path}.html {path}/index.html
        # 2. Les pages à paramètre sont construites une seule fois, sous le nom du
paramètre :
        #    /projects/0192abcd → projects/[id].html. Les « [^/.]+ » n'acceptent ni
barre ni point :
        #    une page déjà trouvée à l'étape 1 (/projects/new.html) n'est pas réécrite.
        #    Une nouvelle page à paramètre dans l'appli ? Ajouter sa règle ici :
        #    infra/web/check-web.mjs fait échouer la construction tant qu'elle manque.
        @project_edit path_regexp ^/projects/[^/.]+/edit$
        rewrite @project_edit /projects/[id]/edit.html
        @project path_regexp ^/projects/[^/.]+$
        rewrite @project /projects/[id].html
        file_server
    }
```

Le jour où l'appli aura une nouvelle page à paramètre (la page d'un membre, en tranche 3b), un oubli dans ce fichier afficherait « introuvable ». Le script `check-web.mjs` lit les pages construites et fait **échouer la construction** tant qu'une règle manque : l'oubli est impossible.

---

4. La CSP : la liste de ce que la page a le droit de faire

Caddy envoie avec chaque page un en-tête **Content-Security-Policy** : la liste de ce que la page peut charger et exécuter. Seulement le code venant de waiggle.io, seulement des appels vers waiggle.io et api.waiggle.io, aucun cadre, aucun code fabriqué à la volée (`eval`). Si un jour un texte piégé arrivait à s'insérer dans une page, le navigateur refuserait d'exécuter son code.

Une CSP stricte casse facilement une appli, sans qu'aucun test ordinaire ne le voie. Deux garde-fous :

- `check-web.mjs` vérifie que chaque petit script écrit dans les pages est autorisé (par son empreinte) ;
- `site-check.mjs` ouvre le site dans un vrai navigateur et signale le moindre blocage.

Le second a trouvé un vrai problème dès son premier essai. Zod, la bibliothèque qui vérifie nos formulaires, essaie de fabriquer du code à la volée pour aller plus vite. La CSP l'en empêchait : Zod s'en passait, mais le navigateur signalait un blocage à chaque page. La solution, dans le tout premier fichier chargé par l'appli :

apps/app/src/strict-browser.ts (extrait)

```
import { z } from 'zod'

z.config({ jitless: true })
```

> **Analogie**
>
> La liste des invités à l'entrée d'une fête : le videur ne laisse entrer que les noms écrits. Même si quelqu'un a réussi à se glisser dans la file, il reste dehors.

---

5. Des images construites par GitHub, vérifiées avant d'être lancées

Le VPS est petit (un seul processeur). Construire l'appli dessus prendrait de longues minutes, pendant que le site tourne. C'est donc GitHub qui construit les **images Docker** : des boîtes toutes prêtes contenant le programme et tout ce qu'il lui faut. Le serveur n'a plus qu'à les télécharger.

Mais comment être sûr qu'une image vient bien de notre CI, à partir de main ? Une étiquette (`ruche-api:<commit>`) peut être remplacée. La relecture de sécurité l'a montré : une pull request venue d'une branche du dépôt pourrait publier une image sous la même étiquette, et contourner ta relecture. D'où deux protections :

infra/deploy-from-pc.sh (extrait)

```
    digest=$(gh attestation verify "oci://$IMAGE_PREFIX-$part:$sha" \
      --repo "$GITHUB_REPO" \
      --signer-workflow "$GITHUB_REPO/.github/workflows/ci.yml" \
      --source-ref refs/heads/main \
      --source-digest "$sha" \
      --deny-self-hosted-runners \
      --format json --jq '.[0].verificationResult.statement.subject[0].digest.sha256')
||
      die "image $part de ${sha:0:7} non vérifiée : la CI de main a-t-elle publié les
images ?"
```

1. **L'attestation de provenance** : une preuve signée par GitHub qui dit « cette image a été construite par tel workflow, sur telle branche, à partir de tel commit ». `gh attestation verify` refuse l'image si l'une de ces informations ne correspond pas.
2. **L'empreinte** (`sha256:…`) : le serveur lance l'image désignée par son empreinte, pas par son étiquette. Une empreinte désigne toujours exactement le même contenu : on lance donc l'image qui vient d'être vérifiée, et aucune autre.

> **Analogie**
>
> Un colis scellé avec le tampon de l'expéditeur. L'étiquette « Waiggle » collée dessus, n'importe qui pourrait l'imiter ; le sceau, non. Et l'on note le numéro du sceau au moment de vérifier : c'est ce colis-là, et pas un autre, qu'on ouvre.

---

6. Le serveur : des portes fermées

Sur le serveur, chaque protection a une raison. D'abord **SSH par clé seulement**. Une clé SSH, c'est une paire : la clé privée reste sur ton PC, la clé publique est posée sur le serveur. Pas de mot de passe à deviner. Et le compte root ne peut plus se connecter du tout :

infra/server/bootstrap.sh (extrait)

```
  cat >"$conf.new" <<EOF
# Écrit par infra/server/bootstrap.sh (étape lock-ssh). Ne pas modifier à la main.
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
AllowUsers $name
X11Forwarding no
EOF
```

Avant d'appliquer ce réglage, `sshd -t` vérifie qu'il est valide. Ensuite, `sshd -T` vérifie qu'il est vraiment appliqué : un fichier de Hostinger qui dirait le contraire est lu après le nôtre, et c'est la première valeur lue qui compte.

Les autres protections :

- **Un pare-feu** : seuls les ports 22 (SSH), 80 et 443 (le site) sont ouverts. La base de données n'est joignable que par l'API, sur un réseau Docker interne.
- **Des conteneurs bridés** : système de fichiers en lecture seule, aucun pouvoir spécial de Linux, Caddy qui ne tourne pas en root, des limites de mémoire et de nombre de programmes.
- **Les mises à jour de sécurité** s'installent seules chaque nuit.

> **Le piège de l'IPv6**
>
> La relecture de sécurité a trouvé un piège sournois. Quand le réseau Docker ne parle pas IPv6, Docker relaie les visiteurs IPv6 en leur donnant à tous **la même adresse**. Or la limitation de débit compte les échecs de connexion par adresse : cinq échecs d'un seul membre auraient bloqué tous les membres en IPv6, c'est-à-dire beaucoup de téléphones en 4G. La correction tient en une ligne (`enable_ipv6: true`), et la procédure contient une vérification sur le vrai serveur.

---

7. Sauvegarder, et surtout savoir restaurer

Chaque nuit à 3 h 30, une **sauvegarde** de la base et des avatars, gardée 14 jours, plus une juste avant chaque déploiement. Une sauvegarde qu'on n'a jamais restaurée ne prouve rien : la CI vérifie donc à chaque pull request le cycle complet. Elle inscrit un premier membre, sauvegarde, en inscrit un second, restaure, puis vérifie que le premier est là et que le second a disparu.

La restauration est prudente. Elle sauvegarde d'abord l'état actuel. Elle restaure ensuite **à côté** de la base en service, et n'échange l'ancienne et la nouvelle qu'une fois tout relu. Si la sauvegarde est abîmée, rien n'a changé.

> **Analogie**
>
> Déménager ses meubles dans la nouvelle maison avant de rendre les clés de l'ancienne. Si le camion tombe en panne en route, on dort encore chez soi.

8. Ce que la relecture de sécurité a trouvé

Trouvaille	Ce qu'on a fait
Tous les visiteurs IPv6 partageaient une même adresse	IPv6 sur le réseau Docker de la porte d'entrée
`deploy.sh` s'arrêtait en erreur quand il n'y avait rien à nettoyer	Nettoyage réécrit, et un essai de trois déploiements en CI
Une image pouvait être remplacée sous la même étiquette	Attestation vérifiée, déploiement par empreinte
Restaurer effaçait la base avant de savoir si la sauvegarde était bonne	Restauration à côté, puis échange
La sauvegarde de la nuit pouvait tomber pendant une restauration	Un verrou : un seul script à la fois
Caddy tournait en root	Un utilisateur à lui, sans aucun pouvoir spécial
En production, les e-mails pouvaient être écrits dans les logs, avec leurs liens de connexion	Refusé : Brevo obligatoire

Les notions de cette leçon

Notion	En une phrase	Où la voir
DNS	L'annuaire qui traduit un nom en adresse	`docs/ops/serveur.md`, étape 2
HTTPS, certificat	La connexion chiffrée, et la preuve d'identité du site	`infra/Caddyfile`
Proxy inverse	Une porte d'entrée qui transmet aux bons services	`infra/Caddyfile`
CSP	La liste de ce qu'une page a le droit de faire	`infra/Caddyfile`
Image Docker		`infra/web/Dockerfile`

Notion	En une phrase	Où la voir
	Un programme emballé avec tout ce qu'il lui faut	
Empreinte	Un identifiant qui désigne un contenu exact	`infra/deploy.sh`
Attestation de provenance	Une preuve signée de l'origine d'une image	`infra/deploy-from-pc.sh`
Clé SSH	Une paire de clés pour se connecter sans mot de passe	`infra/server/bootstrap.sh`
Pare-feu	Ne laisse passer que les ports choisis	`bootstrap.sh`, étape `base`
Sauvegarde testée	Une sauvegarde ne compte que si on sait la restaurer	`infra/stack-test.sh`

Exercices

1. L'annuaire. Dans un terminal, tape `nslookup waiggle.io` puis `nslookup api.waiggle.io`. Les adresses sont-elles les mêmes ? Pourquoi est-ce normal ?

2. La liste des invités. Ouvre waiggle.io, puis les outils de développement du navigateur (F12), onglet « Réseau ». Clique sur la première ligne et trouve l'en-tête `content-security-policy`. Quelle partie autorise les appels vers l'API ?

3. Le colis scellé. Dans `infra/deploy.sh`, trouve la vérification qui refuse une image venant d'un autre dépôt d'images que le nôtre. Que contient la variable `IMAGE_PREFIX`, et d'où vient-elle ?